Practical - 15

Final Integration, Testing, Documentation, GitHub Submission, and Local Deployment

Problem Definition : Integrate all modules into a single deployable StudentHub portal. Verify navigation, responsiveness, authentication, database operations, security checks, AJAX functionality, and documentation. Deploy locally using XAMPP/Apache virtual host and submit GitHub repository.

A. Theory

1. Are all pages properly linked and navigable?

Yes. All 15+ pages are linked seamlessly via a unified navbar.php component. Navigation dynamically shows role-appropriate links (Guest, Student, Admin) and active link highlighting, with breadcrumb buttons to return to dashboards from every sub-page.

2. Are sessions, roles, validations, and database operations working end-to-end?

Yes.

- Sessions & Roles: Users are authenticated with password_verify(), session IDs are regenerated (session_regenerate_id(true)), and roles (admin/student) dictate route permissions.
- Validations: Form validations, duplicate checks (email/enrollment), and file validation (types JPG/PNG/WEBP, size ≤ 2MB) work on both frontend and backend.
- Database CRUD: SELECT, INSERT, UPDATE, and DELETE operations function end-to-end across students, events, registrations, and audit logs.

3. Are security and accessibility checklists completed? Yes.

- Security: SQL Injection prevention (mysqli_real_escape_string), XSS sanitization (htmlspecialchars), Bcrypt password hashing (password_hash), server-side RBAC route guards, file MIME validation, and persistent audit logging.
- Accessibility: Semantic HTML5 elements (<nav>, <main>, <section>), visible :focus outlines for keyboard navigation, high-contrast readable color schemes, and descriptive alt tags on all poster thumbnails.

4. Is project documentation sufficient for installation and testing?

Yes. The README.md and TESTING_CHECKLIST.md provide complete instructions, including XAMPP installation, database import, Apache VirtualHost configuration (studenthub.local), default credentials, and test cases.

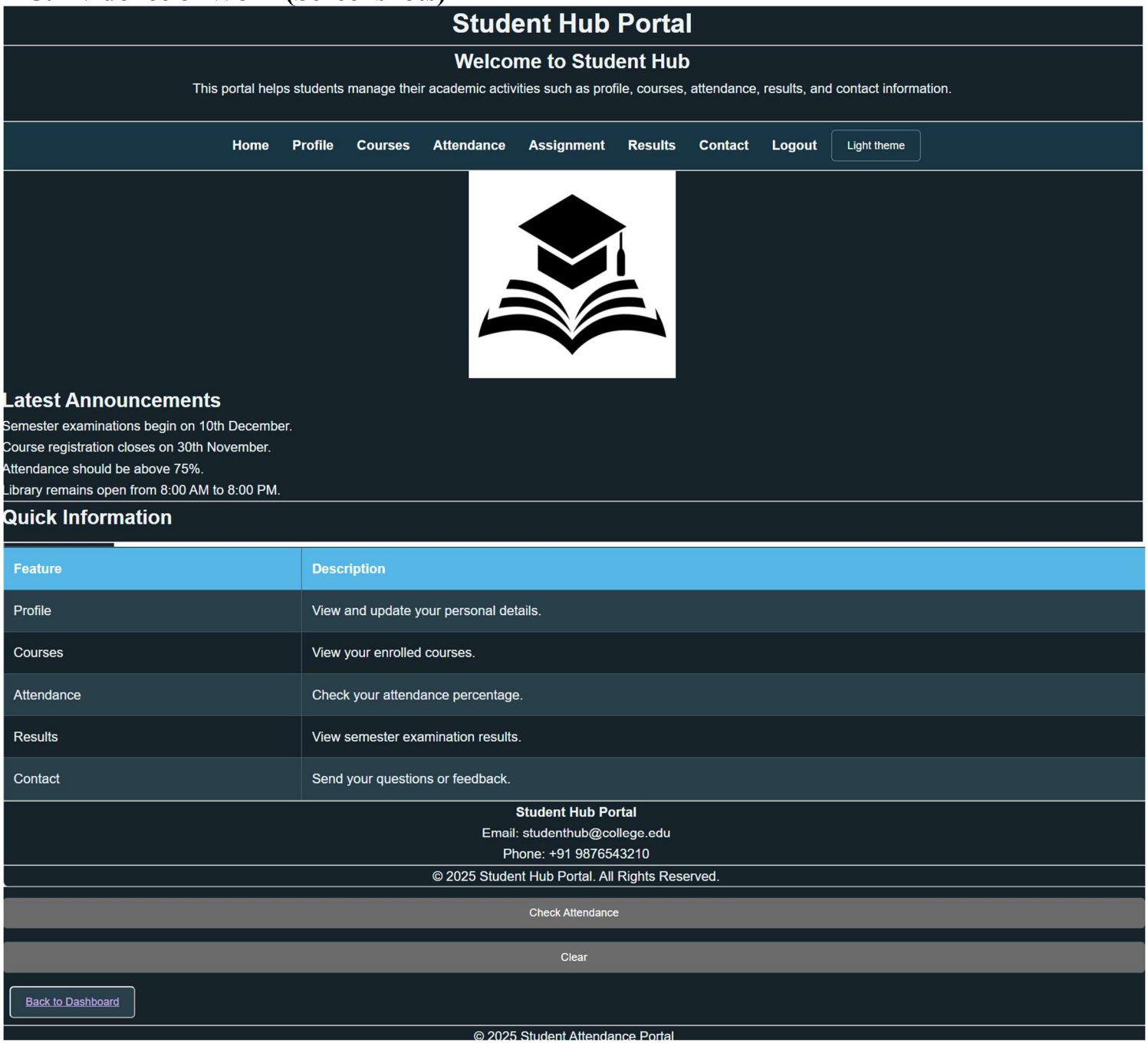
5. Is the project repository clean and properly committed?

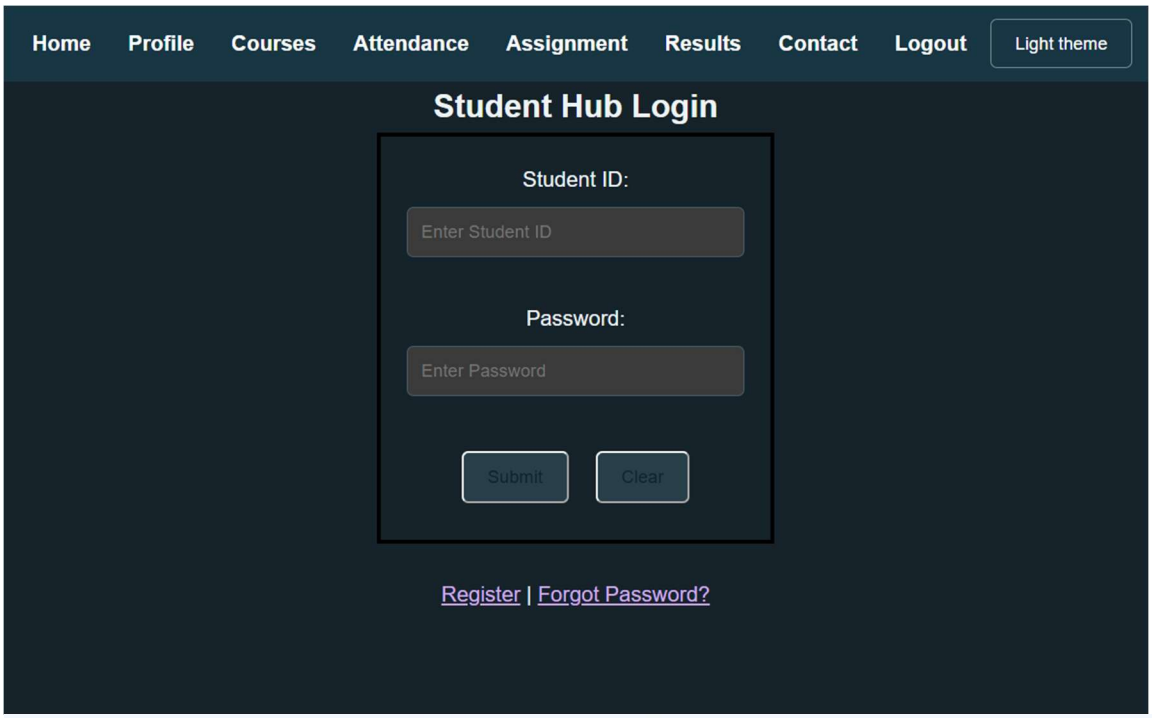
Yes. A .gitignore file was created to ignore .DS_Store and temporary files. All practical files (12, 13, 14, and 15) were staged and cleanly committed to the local main branch with a descriptive commit message: [main 1c05441] feat: complete practical 12, 13, 14, and 15 final integration, testing, and deployment documentation The working tree is completely clean (nothing to commit, working tree clean).

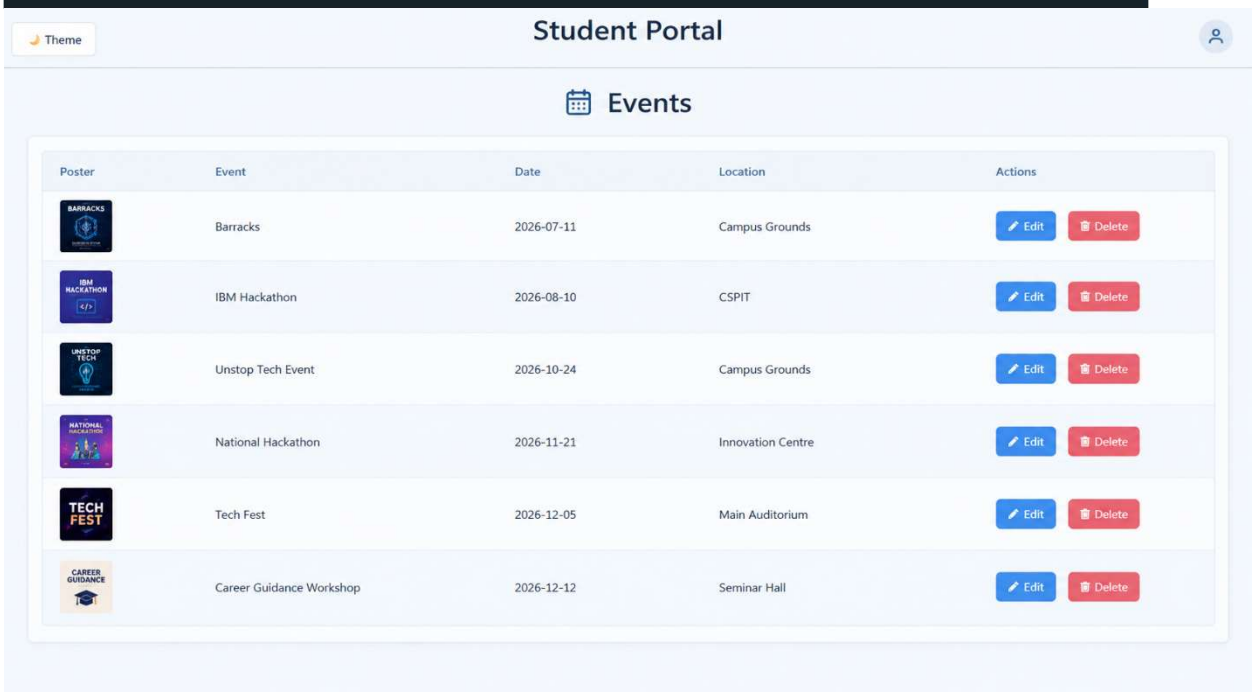
B. Implementation

- Consolidated all separate components (registration, session login, student CRUD, event management with file uploads, JSON API with Fetch API, and role-based admin dashboard) into a unified codebase.
- Standardized file paths, database connections, and session handling using procedural PHP and MySQLi.
- Created deployment assets including vhost_example.conf for Apache virtual hosting and
- studenthub_final.sql for 1-click database initialization.

C. Evidence of Work (Screenshots)




Home
Profile
Courses
Attendance
Assignment
Results
Contact
Logout
Light theme
Student Hub Login
Student ID:
Enter Student ID
Password:
Enter Password
Submit
Clear
Register | Forgot Password?


Theme
Student Portal
Events
Poster
Event
Date
Location
Actions
Barracks
2026-07-11
Campus Grounds
IBM Hackathon
2026-08-10
CSPIT
Unstop Tech Event
2026-10-24
Campus Grounds
National Hackathon
2026-11-21
Innovation Centre
Tech Fest
2026-12-05
Main Auditorium
Career Guidance Workshop
2026-12-12
Seminar Hall
Edit
Delete

D. GitHub Repository Link:

https://github.com/krish-301/Student-Hub-Portal/blob/main/Practical-15/pr%2015.pdf

E. Challenges Faced:

1. Path and Asset Consistency: Integrating standalone practicals required harmonizing relative paths for stylesheets, uploaded posters, and database helpers across all pages.
2. Role Isolation: Ensuring that student users cannot bypass navigation restrictions by manually entering administrative URLs in the address bar.
3. Asynchronous & Synchronous Coexistence: Making sure the traditional PHP CRUD pages and the modern Fetch API single-page interface share the exact same MySQL tables seamlessly without data divergence.

F. Learning Outcomes:

1. Mastered full-stack web integration by uniting discrete frontend, backend, database, and API layers into a cohesive portal.
2. Learned how to configure local Apache Virtual Hosts and manage custom domain routing via the system hosts file.
3. Gained hands-on experience in writing enterprise-grade project documentation, QA verification checklists, and maintaining clean Git version control workflows.

Faculty Remarks: